

PHP Exercise Log | Programming Languages

Exercise 1: Hello World!



The screenshot shows a code editor with the following PHP code:

```
1 <?php
2 echo "Hello, World!"
3 ?>
```

Buttons for Run, Reset, and Solution are visible. The Output panel shows "Hello, World!". The interface is powered by Sphere Engine™.

To open and close a statement, use “<?php” and “?>”

To write a print statement, use “echo”

Exercise 2: Variables and Types



The screenshot shows a code editor with the following PHP code:

```
1 <?php
2 // Part 1: add the name and age variables.
3 $name = "Jake";
4 $age = 20;
5 echo "Hello $name. You are $age years old.\n";
6
7 // Part 2: sum up the variables x and y and
8 // put the result in the sum variable.
9 $x = 195793;
10 $y = 256836;
```

Buttons for Run, Reset, and Solution are visible. The Output panel shows:

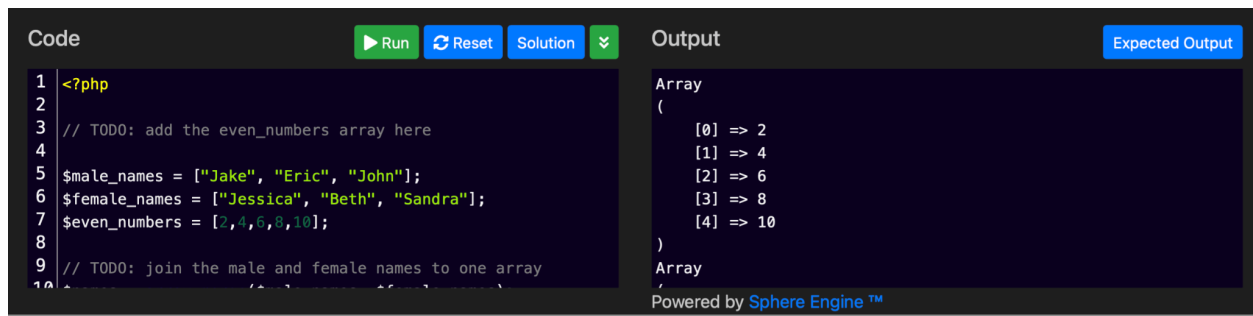
```
Hello Jake. You are 20 years old.
The sum of 195793 and 256836 is 452629.
```

The interface is powered by Sphere Engine™.

There are many types of variables in PHP, but the most common are integers, floats, strings, and booleans (make sure to declare variables before printing)

Variables begin with “\$”

Exercise 3: Simple Arrays



The screenshot shows a code editor with the following PHP code:

```
1 <?php
2
3 // TODO: add the even_numbers array here
4
5 $male_names = ["Jake", "Eric", "John"];
6 $female_names = ["Jessica", "Beth", "Sandra"];
7 $even_numbers = [2, 4, 6, 8, 10];
8
9 // TODO: join the male and female names to one array
10
```

Buttons for Run, Reset, and Solution are visible. The Output panel shows:

```
Array
(
    [0] => 2
    [1] => 4
    [2] => 6
    [3] => 8
    [4] => 10
)
```

The interface is powered by Sphere Engine™.

```
Code Run Reset Solution
8
9 // TODO: join the male and female names to one array
10 $names = array_merge($male_names, $female_names);
11
12 print_r($seven_numbers);
13 print_r($names);
14
15 ?>
```

```
Output Expected Output
)
Array
(
    [0] => Jake
    [1] => Eric
    [2] => John
    [3] => Jessica
    [4] => Beth
    [5] => Sandra
)
Powered by Sphere Engine™
```

Arrays can contain many variables and hold them in a list

Use “\$array_merge” to concatenate arrays

Exercise 4: Arrays with Keys

```
Code Run Reset Solution
2 $phone_numbers = [
3     "Alex" => "415-235-8573",
4     "Jessica" => "415-492-4856",
5     "Eric" => "415-874-7659",
6 ];
7
8 print_r($phone_numbers);
9 ?>
```

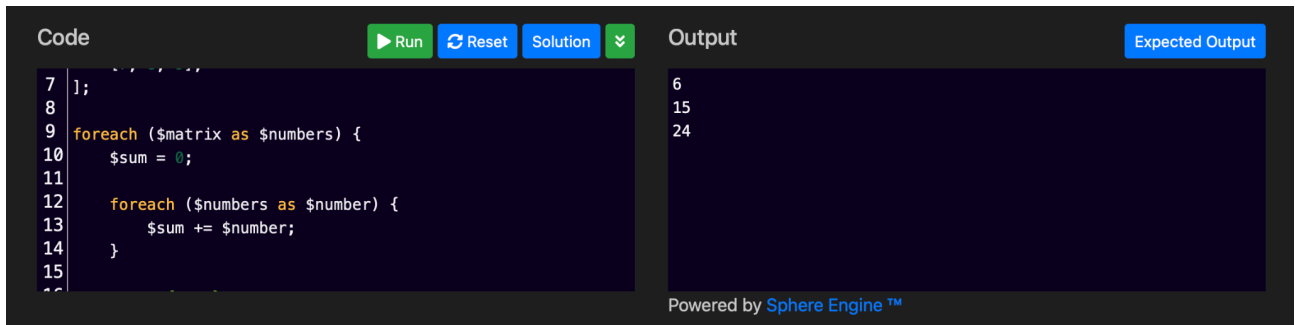
```
Output Expected Output
Array
(
    [Alex] => 415-235-8573
    [Jessica] => 415-492-4856
    [Eric] => 415-874-7659
)
Powered by Sphere Engine™
```

```
Code Run Reset Solution
3     "Alex" => "415-235-8573",
4     "Jessica" => "415-492-4856",
5 ];
6
7 $phone_numbers ["Eric"] = "415-874-7659";
8
9 print_r($phone_numbers);
10 ?>
```

```
Output Expected Output
Array
(
    [Alex] => 415-235-8573
    [Jessica] => 415-492-4856
    [Eric] => 415-874-7659
)
Powered by Sphere Engine™
```

Couple ways to add name and number to array list: add directly to array list (array definition) or as a separate line of code

Exercise 5: Multidimensional arrays



```
Code Run Reset Solution
```

```
7 1;  
8  
9 foreach ($matrix as $numbers) {  
10     $sum = 0;  
11  
12     foreach ($numbers as $number) {  
13         $sum += $number;  
14     }  
15  
16 }
```

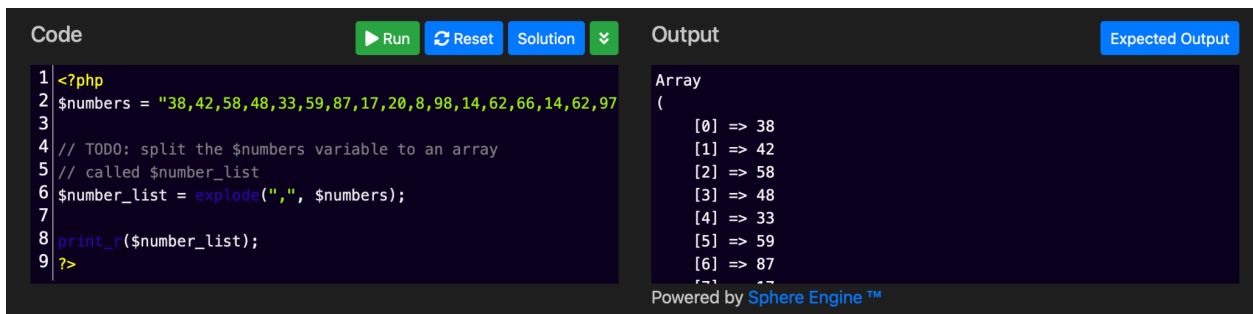
```
Output Expected Output
```

```
6  
15  
24
```

Powered by Sphere Engine™

Confused on the second “foreach” statement

Exercise 6: Strings



```
Code Run Reset Solution
```

```
1 <?php  
2 $numbers = "38,42,58,48,33,59,87,17,20,8,98,14,62,66,14,62,97";  
3  
4 // TODO: split the $numbers variable to an array  
5 // called $number_list  
6 $number_list = explode(",", $numbers);  
7  
8 print_r($number_list);  
9 ?>
```

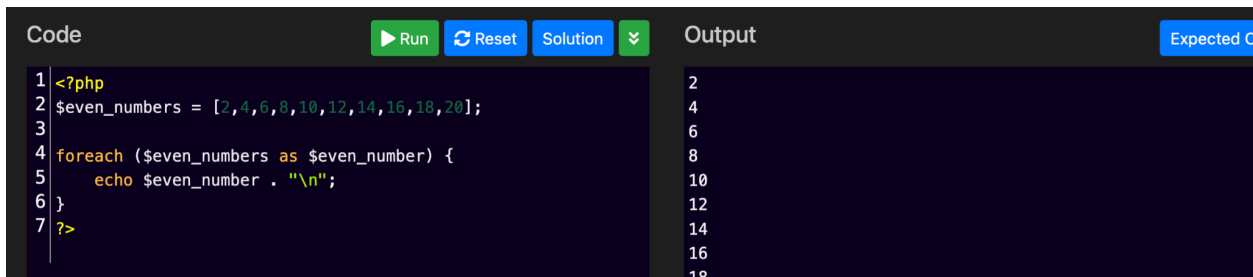
```
Output Expected Output
```

```
Array  
(  
    [0] => 38  
    [1] => 42  
    [2] => 58  
    [3] => 48  
    [4] => 33  
    [5] => 59  
    [6] => 87  
    [7] => 17  
    [8] => 20  
    [9] => 8  
    [10] => 98  
    [11] => 14  
    [12] => 62  
    [13] => 66  
    [14] => 14  
    [15] => 62  
    [16] => 97  
)
```

Powered by Sphere Engine™

Create a new array “\$number_list” and use the new *explode* function to split array

Exercise 7: For Loops (& Foreach loops)



```
Code Run Reset Solution
```

```
1 <?php  
2 $even_numbers = [2,4,6,8,10,12,14,16,18,20];  
3  
4 foreach ($even_numbers as $even_number) {  
5     echo $even_number . "\n";  
6 }  
7 ?>
```

```
Output Expected Output
```

```
2  
4  
6  
8  
10  
12  
14  
16  
18
```

Using “foreach” loop!

Exercise 8: While Loops

Code

Run

Reset

Solution

⌵

```
1 <?php
2 $numbers = [56, 65, 26, 86, 66, 34, 78, 74, 67, 18, 34, 73, 4
3
4 // TODO: Print odd numbers only
5 $index = 0;
6 while( $index < count( $numbers ) )
7 {
8     $number = $numbers[ $index ];
9     ++$index;
10 }
```

Output

65
67
73
45
67
75
23
3
73
^

Powered by Sphere Engine™

Code

Run

Reset

Solution

⌵

```
9     ++$index;
10
11     if( $number % 2 == 0 )
12         continue;
13
14     echo "$number\n";
15 }
16 ?>
```

Output

65
67
73
45
67
75
23
3
73
^

Powered by Sphere Engine™

Question: Kinda confused about the way “While” loops are written

Exercise 9: Functions

There are two types of functions – library functions and user functions. Library functions (like *array_push*) can be used by anyone, probably most familiar

Code

Run

Reset

Solution

⌵

```
1 <?php
2 // Write the function squared_sum here
3 function squared_sum($numbers){
4     $squared_sum = 0;
5
6     foreach ($numbers as $number){
7         $squared_sum += $number * $number; // sqrd operation
8     }
9
10 }
```

Output

382629

Powered by Sphere Engine™

Code

Run

Reset

Solution

⌵

```
7     $squared_sum += $number * $number; // sqrd operation
8   }
9
10  return $squared_sum;
11 } // closes the brakes for function
12 echo squared_sum([56, 65, 26, 86, 66, 34, 78, 74, 67, 18, 34
13
14 ?>
```

Output

Exp

```
382629
```

Powered by [Sphere Engine™](#)

Exercise 10: Objects

Fun fact: PHP is an object-oriented language

Classes define how objects behave; *class* is the definition of an object

Most important part of object-oriented programming is inheritance. Allows us to reuse code and extend it.

<< Previous Tutorial

Next Tutorial >>

Code

▶ Run

↺ Reset

Solution

⌵

```
1 <?php
2 class Car {
3     public function __construct($brand, $year) {
4         $this->brand = $brand;
5         $this->year = $year;
6     }
7
8     public function print_details() {
9         echo "This car is a " . $this->year . " " . $this->b
10
```

Output

Expected Output

This car is a 2006 Toyota.

Powered by Sphere Engine™

Code

Run

Reset

Solution

```
7  
8     public function print_details() {  
9         echo "This car is a " . $this->year . " " . $this->b  
10    }  
11 }  
12  
13 $car = new Car("Toyota", 2006);  
14 $car->print_details();
```

Output

Expected Output

This car is a 2006 Toyota.

Powered by Sphere Engine™